Deep Reinforcement Learning for Autonomous Robot Navigation

From Simulation to Real-World Deployment

Dr. Wei Chen[1]   Prof. Yuki Nakamura[2]

[1]MIT CSAIL

[2]University of Tokyo

International Conference on Robotics and Automation
May 2025

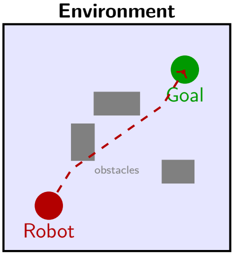
Outline

1. Introduction
2. Background
3. Methodology
4. Results
5. Conclusion

The Challenge of Autonomous Navigation

Key challenges:

- Complex, dynamic environments
- Partial observability from onboard sensors
- Real-time decision making under uncertainty
- Safe exploration during training
- Sim-to-real transfer gap



Our approach:
Use deep reinforcement learning with domain randomization and curriculum learning to train navigation policies in simulation, then transfer to real robots.

Reinforcement Learning Basics

Markov Decision Process (MDP)

An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: state space $\mathcal{A}$: action space
- $P(s'|s, a)$: transition dynamics
- $R(s, a)$: reward function γ: discount factor

Objective

Find policy π^* that maximizes expected cumulative reward:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\right]$$

Key Insight

Policy Gradient Methods

Proximal Policy Optimization (PPO) is our algorithm of choice:

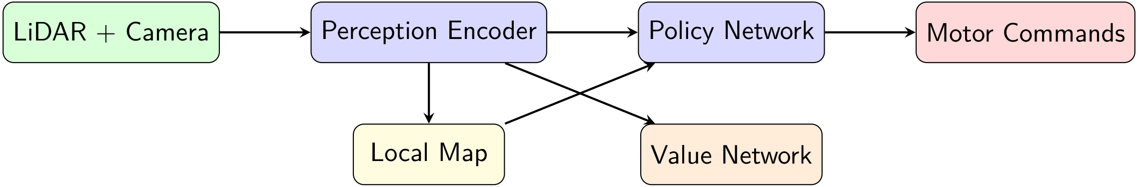
$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio.

Advantages of PPO:

1. Stable training with clipped objective
2. Sample efficient with multiple epochs per batch
3. Works well with both continuous and discrete actions
4. Easy to parallelize across environments

System Architecture



- **Perception Encoder**: CNN processes LiDAR scans and depth images
- **Local Map**: Occupancy grid built from sensor history
- **Policy Network**: Outputs velocity commands (v, ω)
- **Value Network**: Estimates state value for advantage computation

Training Code

```python
import torch
from stable_baselines3 import PPO
from navigation_env import NavigationEnv

# Create vectorized training environments
env = make_vec_env(NavigationEnv, n_envs=16, env_kwargs={
    "map_size": 10.0,
    "max_obstacles": 20,
    "domain_randomization": True,
})

# Configure PPO with custom network
model = PPO(
    "MultiInputPolicy",
    env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=256,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    verbose=1,
    tensorboard_log="./logs/",
)

# Train for 10M timesteps with curriculum
model.learn(total_timesteps=10_000_000,
            callback=CurriculumCallback())
```

Simulation Results

Table: Navigation performance across environment difficulties

Method	Success (%)	Collision (%)	Avg. Time (s)
Bug2 Algorithm	62.3	15.2	34.7
RRT*	78.1	8.4	28.3
DWA Planner	71.5	12.1	31.2
Vanilla PPO	81.4	6.7	22.8
SAC	83.2	5.9	21.5
Ours (PPO+DR+CL)	**94.7**	**2.1**	**18.3**

- DR = Domain Randomization, CL = Curriculum Learning
- Evaluated on 1000 randomly generated environments
- Our method achieves **94.7%** success rate with only **2.1%** collisions

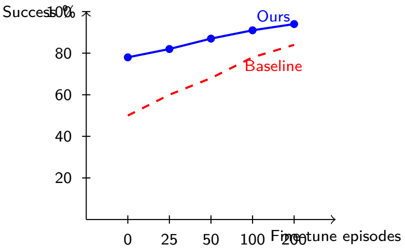
Sim-to-Real Transfer

Real-world experiments:

- TurtleBot3 Waffle Pi platform
- Hokuyo LiDAR + Intel RealSense
- 5 different indoor environments
- 50 trials per environment

Key findings:

1. Zero-shot transfer: 78% success
2. With 100 real episodes fine-tuning: 91% success
3. Average planning time: 12ms (real-time)



Summary and Future Work

Key contributions:

1. Novel deep RL framework for autonomous navigation combining PPO with domain randomization and curriculum learning
2. State-of-the-art simulation performance: **94.7%** success rate
3. Successful sim-to-real transfer with **91%** real-world success after minimal fine-tuning
4. Real-time inference at **12ms** per decision on embedded hardware

Future directions:

- Multi-robot coordination and formation control
- Outdoor navigation with GPS-denied environments
- Integration with semantic scene understanding
- Formal safety guarantees via constrained RL

Thank you! Questions?

`weichen@mit.edu` | `github.com/wchen/nav-rl`

References I

J. Schulman et al., "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

J. Tobin et al., "Domain randomization for transferring deep neural networks from simulation to the real world," in *IROS*, 2017.

V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *IROS*, 2017.